

Analysis: Double or One Thing

For any string S , the number of new strings we can obtain from it is at most $2^{|S|}$ because for each character in S , there are 2 choices: to highlight it or not.

Test Set 1

Since the length of S is at most 10, the number of new strings we can obtain is at most 2^{10} . We can enumerate all of them to find the lexicographically smallest one. The time complexity of this solution is $O(2^{|S|} \times |S|)$ because there are $2^{|S|}$ strings in total to compare, and the length of them is at most $2 \times |S|$.

Test Set 2

Now that the length of S can be up to 100, 2^{100} is too large to enumerate all of them.

Note that a string p appears before a different string q in alphabetical order if p is a prefix of q or if p has a letter lexicographically smaller at the leftmost position at which p and q differ.

Then, for any character S_i in S , we have the following rule to decide whether to highlight it or not:

- If the next different character in S is lexicographically larger than the current character (in other words, there is an index j that $S_j > S_i$ and $S_k = S_i$ for all $i < k < j$), then we must highlight S_i . That is because when we double S_i , we also push S_j right and replace that index with S_i at the same time. Therefore doubling S_i will make the new string lexicographically smaller.
- If the next different character in S is lexicographically smaller than the current character (in other words, there is an index j that $S_j < S_i$ and $S_k = S_i$ for all $i < k < j$), then we must NOT highlight S_i . That is because when we double S_i , we also push S_j right and replace that index with S_i at the same time. Therefore doubling S_i will make the new string lexicographically larger.
- If there is no next characters different from S_i (in other words, i is the last index in S or $S_k = S_i$ for all $i < k < |S|$), then we must NOT highlight S_i . That is because doubling S_i will make the original string be the prefix of the new string, which means the new string is lexicographically larger.

There are a lot of different ways to implement it and here is one: We can preprocess the given S into groups of same and continuous characters. For example, `BOOKKEEPER` is preprocessed as $[(B, 1), (O, 2), (K, 2), (E, 2), (P, 1), (E, 1), (R, 1)]$. Then for each element in this list, we output the character with twice its original occurrence if the character of the next element is lexicographically larger than it. Otherwise, we output the character with its original occurrence.

The time complexity of this solution is $O(|S|)$ because we need to iterate through S when preprocessing it, and iterate through the preprocessed list with length up to $|S|$ when outputting the final answer.

Analysis: Equal Sum

In this problem we first pick half of the input, then the judge picks the other half, and then we need to solve an [NP-complete problem](#) on that input. This means we have to be really strategic on how we pick that input.

We only get to provide half of the numbers. So, it might be best to first see how good of a split we can do with using just the judge provided numbers and only afterwards come up with the numbers we provide.

A common heuristic to solve problems in which we do not know all of the input yet is to optimize locally. In this case, sort out the numbers into two sets, trying for their sums to be as close as possible. That means, we go through the input integers and assign them to the set that has the smallest current sum. After we process all of the judge's integers, this guarantees that the difference between the sets is bounded by the size of a single integer (10^9), as opposed to being bounded by the total sum of the integers we have seen (10^{11}).

We need our integers to always be able to make up for that difference, so we need to guarantee at least some of the integers we pick are big enough. Moreover, we need to be able to make up for the difference exactly, so we want to pick integers that get progressively more precise. This smells like binary. However, when writing in binary we get to sum over a subset of the powers of 2. In this case, the integers that do not end up in one subset go to the other one, so binary can just write any possible difference. That being said, powers of 2 still work, as long as we do not write in binary.

After we finish sorting out the judge's input, we know that the difference between the two subsets is at most 10^9 . Therefore, if we decide where to put the largest power of 2 (2^{29}) using the same algorithm, that guarantees that the difference is now at most 2^{29} . As long as the next number is always at least half of the previous one, we can maintain the difference bounded by the last processed number. If we process the powers of 2 last, and in decreasing order, the difference between the two subsets at the end is no larger than $2^0 = 1$. Since the limits guarantee that the total sum is even, the difference is 0.

The paragraph above shows that any set of integers up to X with even sum that contains every power of 2 no greater than X can be partitioned into two subsets of equal sum. Moreover, the partition can be found efficiently. There are 30 powers of 2 between 1 and 10^9 , which means we can choose the other 70 integers we are entitled in any way.

Analysis: Weightlifting

Test Set 1

The limits for Test Set 1 are small enough that we can afford to use brute force. Since there is no reason to have more than $\max_j \mathbf{X}_{i,j}$ weights of type i , the number of possible states of the stack to consider is very limited (in fact, it is $\sum_{0 \leq a,b,c \leq 3} \frac{(a+b+c)!}{a!b!c!} = 5248$). To solve for the minimum number of operations to finish all of the exercises, you can use a [breadth-first search](#) over all possible combinations of the stack states and the number of completed exercises. Specifically, the breadth-first search starts at the state of an empty stack with no completed exercises, and the final answer is the distance from that to the state where the stack is empty but all exercises are completed.

Test Set 2

The first observation we can make to solve Test Set 2 of this problem is the following: if there are any weights that are used in every exercise, it is always optimal to put them at the bottom of the stack before the first exercise, and leave them there until after the last one. Moreover, if there are multiple such weights, putting them in any order is equivalent. Formally, if C is the multiset of weights that is common to all exercises, there is an optimal solution that has C at the bottom of the stack during all exercises.

Now, for a given full sequence of exercises, let us call A to the largest sequence of weights that appears at the bottom of the stack for all exercises (A could be empty). Since A is used for all exercises, $A \subseteq C$. If we use the first observation, then also $C \subseteq A$, so $A = C$. Therefore, we can make a second observation: if there is more than one exercise, there is at least one time in between exercises in which the stack contains exactly C . This points towards a divide and conquer solution: find that intermediate point and then recursively solve the problem of minimizing the operations before and after that point, disregarding C .

As it is common with divide and conquer approaches, we can either find the place to split without recursion, or we can use [memoization](#) to be able to simply try every possible split point, without incurring significant additional computation time.

To formalize the approach, let $C(\ell, r)$ be the multiset of weights that is the intersection of the multiset of weights needed for each exercise between ℓ and r , inclusive. Additionally, let $M(\ell, r)$ be the minimum number of operations to get from a stack containing $C(\ell, r)$, in any order, perform all operations between ℓ and r , inclusive, and leave the stack with $C(\ell, r)$ on it, in the same order. To calculate $M(\ell, r)$ we follow the strategy above: for some mid-point x we do this twice: load the additional weights for one part of the split, optimize according to a recursive result, then unload those additional weights. The additional weights for the left part of the split are $C(\ell, x) \setminus C(\ell, r)$ and for the right part of the split are $C(x+1, r) \setminus C(\ell, r)$ (notice that $C(\ell, r)$ is included in the other two multisets by definition). Therefore, the cost in addition to the recursion for the mid-point x is $2 \times (|C(\ell, x)| + |C(x+1, r)| - 2 \times |C(\ell, r)|)$. Putting it all together:

- $M(\ell, r) = 0$, if $\ell = r$;
- $M(\ell, r) = \min_{\ell \leq x < r} (M(\ell, x) + M(x+1, r) + 2 \times (|C(\ell, x)| + |C(x+1, r)| - 2 \times |C(\ell, r)|))$, otherwise.

Precomputing the $O(\mathbf{E}^2)$ values of C takes $O(\mathbf{E}^2 \times \mathbf{W})$ time. With memoization, we can compute the $O(\mathbf{E}^2)$ values M in $O(\mathbf{E}^3)$ overall time. This is fast enough to pass Test Set 2.